



AI Infrastructure

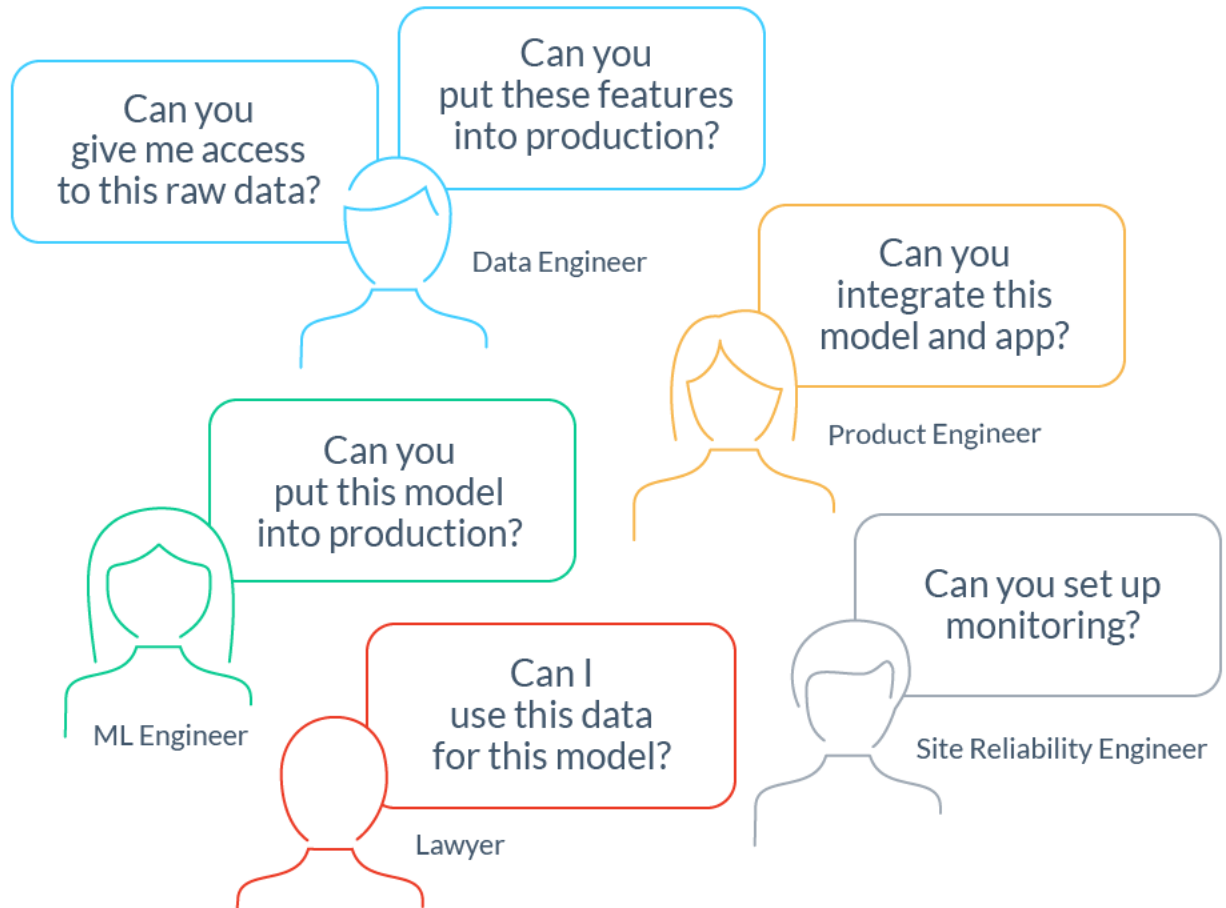
Ioannis Papapanagiotou

Introduction to MLOps

Why We Need DevOps for ML Data?



- ✓ Build
- ✓ Train
- ✓ Deploy



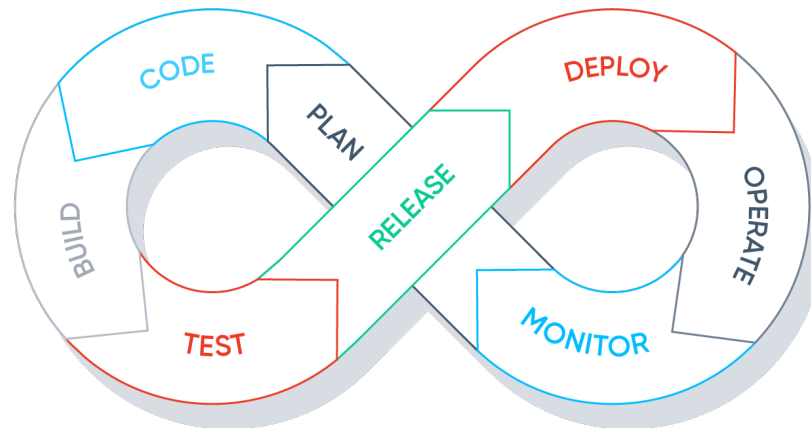
Remember when releasing software was slow and painful?

Developing and deploying software 20 years ago looked a lot like developing ML applications today:

- feedback loops were incredibly long and by the time you released an application, the requirements and designs that you started with were obsolete.
- Then, in the late 00's, [DevOps](#) emerged as a set of software engineering best practices to manage the software development lifecycle and enable continuous, rapid improvement.

DevOps

- Under the DevOps approach, software engineers work in a well-defined shared codebase.
- Once an incremental change is ready for deployment, the engineer checks it into a version control system.
- A Continuous Integration/Continuous Delivery (CI/CD) pipeline
 - picks up the latest changes,
 - runs unit tests,
 - builds the software artifacts,
 - runs integration tests, and
 - eventually rolls out the change to production in a controlled fashion or stages the release for distribution.

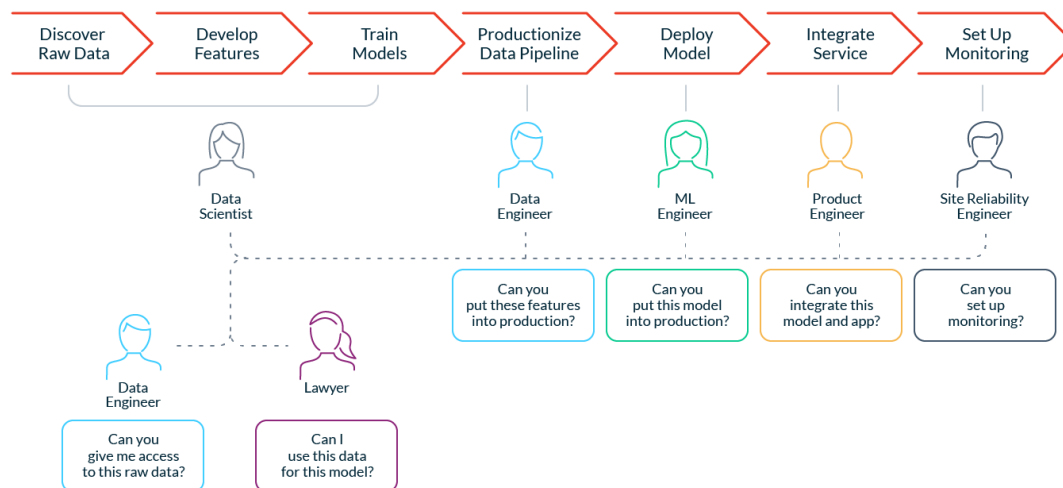


Key Benefits of DevOps

- **Software engineers own their code end-to-end.** They're empowered and ultimately responsible for every line of code that's running in production. This sense of ownership generally increases code quality, as well as application availability and reliability.
- **Teams are able to iterate rapidly** and aren't slowed down by months-long waterfall cycles. Instead, they can test new capabilities with real customers almost instantly.
- **Performance and reliability issues can be identified and addressed quickly.** If a health metric drops right after the most recent deployment, rollbacks are auto-triggered, and the code change that triggered the deployment is a likely culprit.

Deploying ML is still slow

- In stark contrast to software engineering, data science doesn't have a well-defined, fully automated process to get into production quickly. The process to build an ML application and deploy it in a product involves multiple steps:
- *Data scientists need to coordinate their work across teams and disciplines*



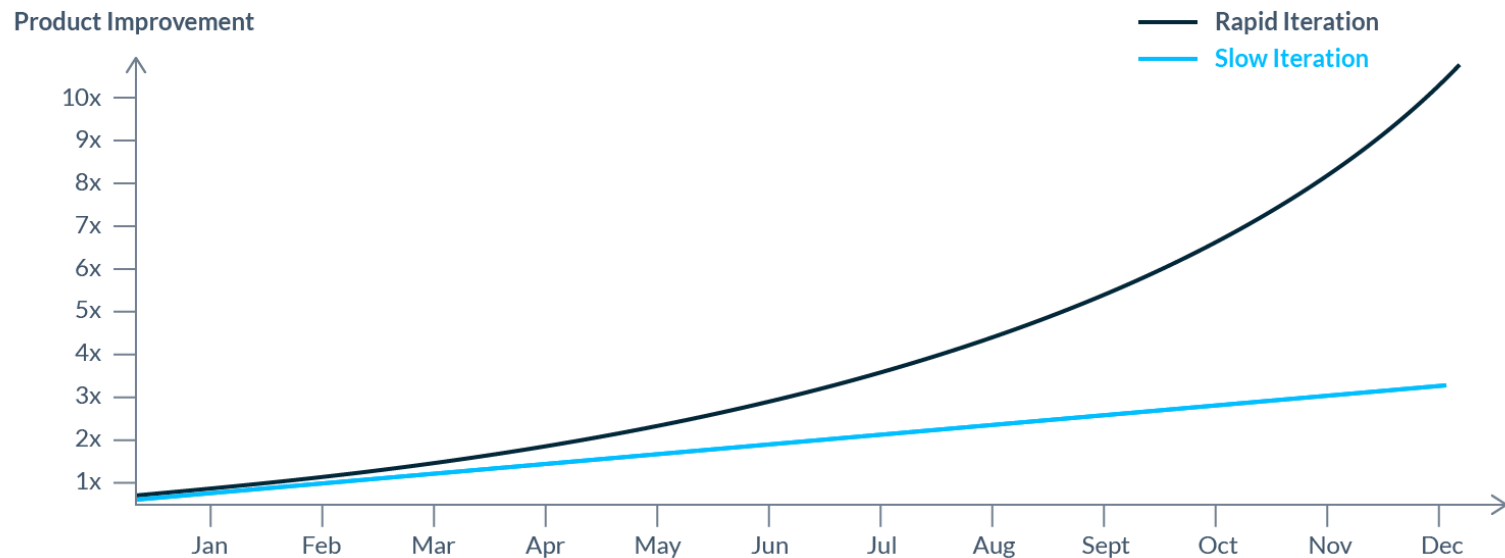
ML Engineering processes

- **Discover and access the raw data.** Data scientists in most companies spend up to 80% of their time searching for the right raw data for their modeling problem. This often requires cross-functional coordination with data engineers and regulatory teams.
- **Develop features and train models.** Once data scientists get access to the data, they often spend weeks cleaning and turning it into features and labels. They then train models, evaluate them, and repeat the process several times.
- **Productionize data pipelines.** Data scientists then turn to engineering teams to “productionize” their feature data pipelines. This typically means “throwing feature transformation code over the wall” for an efficient reimplementation that is production-ready (more on this later).
- **Deploy and integrate the model.** This step typically involves integrating with a service that uses the model for predictions. For example: an online retailer’s mobile app that uses a recommendation model to predict product offers.
- **Set up monitoring.** Again, engineering help is required to ensure that the ML model and data pipelines continue to operate correctly.

Rapid vs Slow Iteration

- As a result, ML teams face the same types of problems that software engineers dealt with 20 years ago:
 - **Data scientists lack full ownership** of the model and feature lifecycle. They depend on others to deploy their changes and maintain them in production.
 - **Data scientists are unable to iterate rapidly.** The lack of ownership makes it impossible to iterate quickly. Teams on which data scientists depend have their own priorities and agendas, which often introduce delays and uncertainty. Iteration speed is critical and delays can easily compound to levels that squash productivity:

Rapid vs. Slow Iteration – 1 year



MLOps Definition

ML Ops is an ML engineering culture and practice that aims at unifying **ML Development** (Dev) and **ML Operations** (Ops)

MLOps vs DevOps

In ML systems:

- Continuous Integration (CI) is not only about testing and validating code and components, but also testing and validating data, data schemas, and models.
- Continuous Delivery (CD) is not only about a single software package or a service, but a system (an ML training pipeline) that should automatically deploy another service (model prediction service).
- Continuous Training (CT) is a new property, unique to ML systems, that's concerned with automatically retraining candidate models for testing and serving.
- Continuous Monitoring (CM) is not only about catching errors in production systems, but also about monitoring production inference data and model performance metrics tied to business outcomes.

MLOps

- The common challenge encountered in a typical enterprise scenario is that the ML models worked in lab environment will remain stay at the proof-of-concept stage in many cases.
 - If the model is rolled out into production, it becomes stale due to frequent source data changes that requires rebuilding of model.
- As the models are retrained multiple times, it is required to keep track for model performance and corresponding features and hyperparameters that are used for retraining the model.
- To perform all these operations, there should be a well-defined reproducible process in-place to implement the end-to-end machine learning operations (MLOps) that keeps the model current and accurate in production environment.

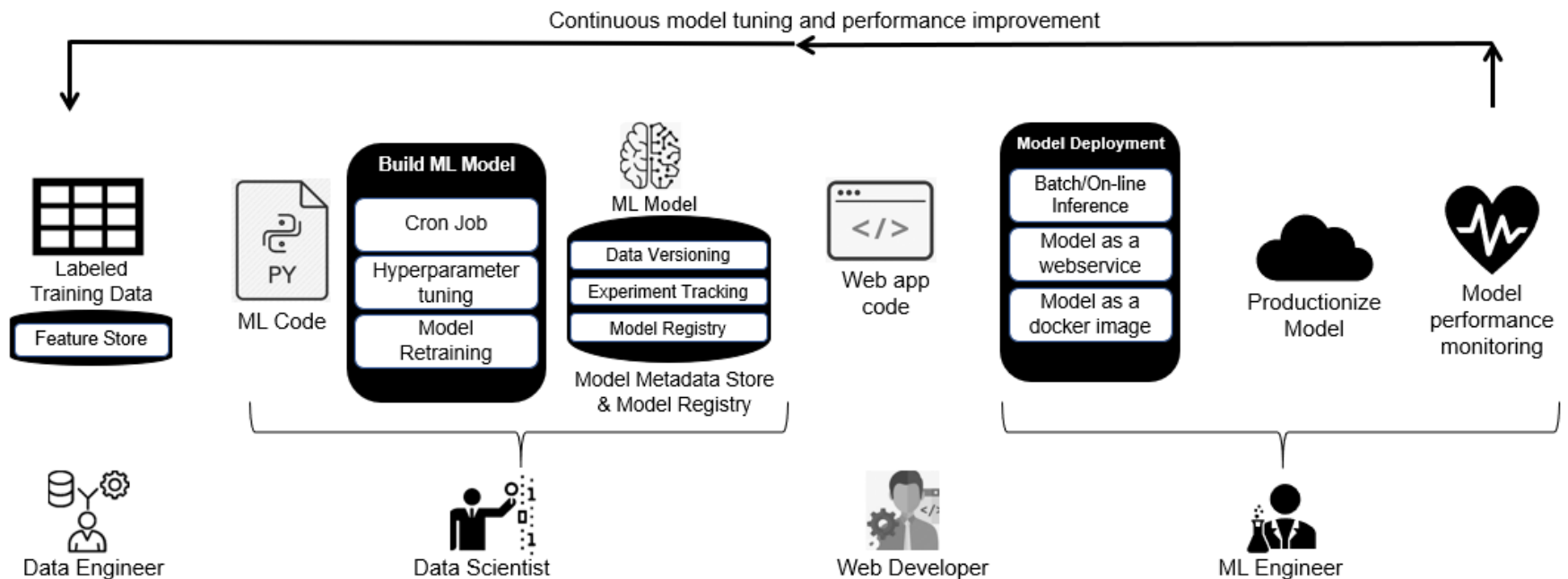
ML Ops

Automation and **Monitoring** at all steps of system construction including:

- integration
- testing
- releasing
- deployment
- infrastructure management

ML model Operations Lifecycle

- We cover the entire process of model development to model deployment to model performance monitoring in a seamless manner.



Model Retraining

- The model retraining is very important at regular intervals, say fortnightly or monthly or quarterly or on-demand basis as it is very likely that the underlying source data will change over a period in the real-world scenario.
- A cron job is scheduled to retrain the model at the predefined intervals, or as and when the source data is changed, or as and when the model performance is degraded.
- There could be some changes to the model code due to hyperparameter tuning during model re-training.
- It is required to automate these model operations by having an automated model pipeline.

References

- [Key requirements for an MLOps Foundation \(Google Cloud\)](#)
- [DeepLearning.AI LLMOps Introduction](#) → for hands on lab on BigQuery